



Access Management Page — Technical Documentation

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Access Management Page — Technical Documentation	1
Overview	3
Registration & Discovery	3
Core Modules	4
Database Tables	6
Permission Hierarchy (runtime)	6
Managed Pages (UI scope)	8
Tab 1 — Users & Roles	8
Tab 2 — Page Permissions	9
Database Health Check	11
Session State Keys	12
Error Handling	12
Logging	12
Relationship to Other Admin UIs	13
Admin Capability Matrix	13
Fallback Admins (DB unavailable)	14
Dependency Graph	14

Application: Market Data Portal (MDP)

Source: `app/pages/access_management.py`

URL path: `/access_management`

Purpose: Centralized Identity and Access Management (IAM) for **system roles** (`coreiq_user_roles`) and **page-level permissions** (`coreiq_page_access_control`).

Overview

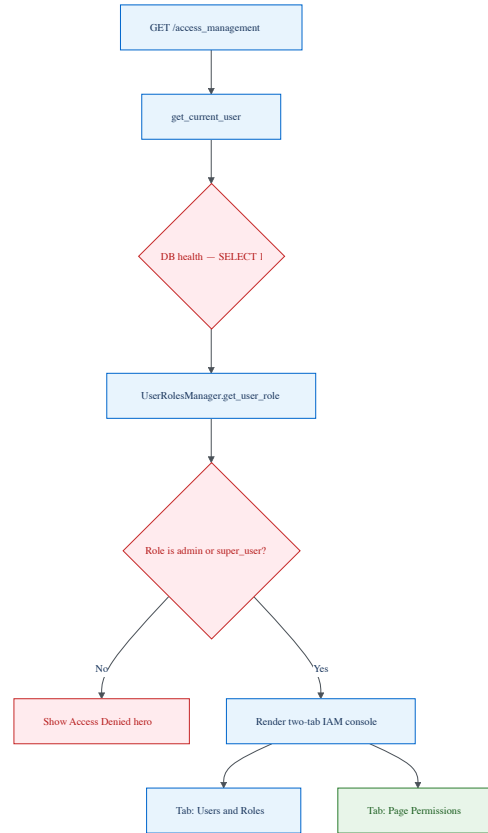
Access Management is the administrative console for who can use restricted portal pages and what **system role** each user holds. Only users with role `admin` or `super_user` may open the page. Within the page, **admins** have additional powers (create admin users, grant delete permission, delete users).

Registration & Discovery

Property	Value
<code>main.py</code> registration	Yes — <code>url_path=access_management</code>
Footer link	Shown when <code>_can_access_mgmt(user_email)</code> is true
Auth	<code>get_current_user()</code> — identity via <code>main.py</code> cookie hydrate

Gate: only `admin` or `super_user` roles reach the Identity and Access Management (IAM) console; others see an access-denied hero.

Gate: only admin or superuser roles reach the Identity and Access Management (IAM) console; others see an access-denied hero.



Core Modules

AccessControlManager (`app/core/access_control.py`)

Manages `coreiq_page_access_control` — per-page, per-user permission rows.

Method	Purpose
<code>check_access(page, email, required_permission)</code>	Runtime gate used by feature pages
<code>get_page_users(page)</code>	List Access Control List (ACL) rows for a page
<code>set_user_access(page, email, permission, created_by)</code>	Upsert Access Control List (ACL) row
<code>remove_user_access(page, email)</code>	Delete Access Control List (ACL) row
<code>get_permission_options()</code>	All valid permission strings

`UserRolesManager` (same file)

Manages `coreiq_user_roles` — global role per email.

Method	Purpose
<code>get_user_role(email)</code>	Returns <code>admin</code> , <code>super_user</code> , <code>user</code> , or None
<code>get_all_users()</code>	Full user list ordered by role hierarchy
<code>set_user_role(email, role, display_name, created_by)</code>	Upsert user
<code>delete_user(email)</code>	Remove user (admin-only in UI)
<code>is_admin</code> / <code>is_admin_or_super_user</code>	Convenience checks

Database Tables

coreiq_page_access_control

Column	Purpose
page_name	Page identifier (e.g. retailer_adding)
user_email	Lowercased email
permission_level	See hierarchy below
created_by	Granting admin email
created_at / updated_at	Timestamps

Upsert: INSERT ... ON DUPLICATE KEY UPDATE permission_level, updated_at

coreiq_user_roles

Column	Purpose
user_email	PK / unique
display_name	Optional friendly name
role	admin super_user user
created_by	Who added the user
created_at / updated_at	Timestamps

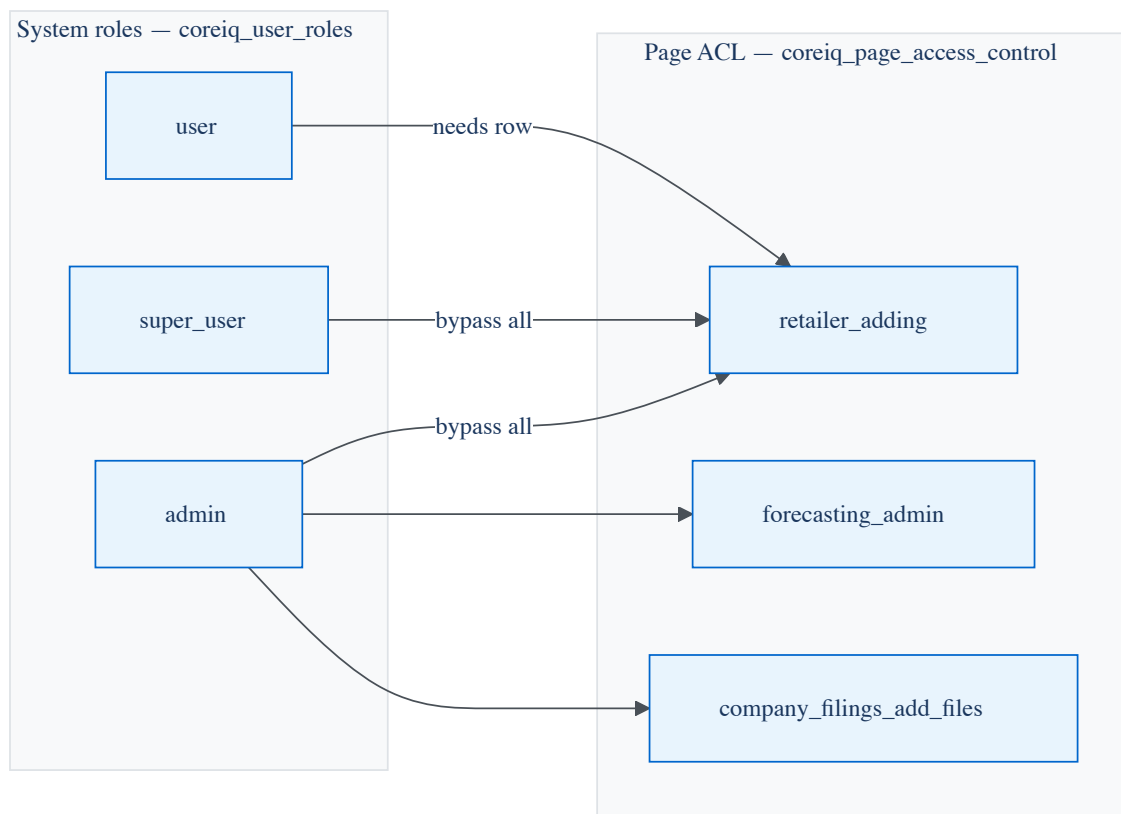
Permission Hierarchy (runtime)

Used by AccessControlManager.check_access :

Level	Numeric rank	Meaning
<code>view_only</code>	1	Read-only
<code>upload_only</code>	2	Upload capability
<code>edit</code>	3	Edit content
<code>delete_admin</code> , <code>allow</code>	4	Full page access
<code>deny</code>	—	Explicit block (always false)

Bypass: Users with role `admin` or `super_user` return `True` for any `check_access` call without reading Access Control List (ACL) table.

Bypass: Users with role admin or superuser return True for any checkaccess call without reading Access Control List (ACL) table.



Managed Pages (UI scope)

Configured in `MANAGED_PAGES` :

page_name	Display label
<code>company_filings_add_files</code>	Company Filings File Manager
<code>retailer_adding</code>	Retailer Adding
<code>forecasting_admin</code>	Forecasting Admin

Note: The public **forecasting viewer** (`forecasting` page) uses separate Access Control List (ACL) checks in `forecasting.py` but is **not** listed in Access Management UI.

Tab 1 — Users & Roles

Stats grid

Total users, Admins, Super Users, standard Users — computed from `get_all_users()` .

Add / Update User form

Field	Widget
Email	<code>st.text_input</code> key <code>am_u_email</code>
Display name	<code>am_u_name</code>
Role	<code>am_u_role</code> selectbox
Submit	Add User → <code>UserRolesManager.set_user_role</code>

Rules:

- Only **admins** can assign role `admin` (super_users blocked with error).
- Mutations disabled when `db_ok=False` .

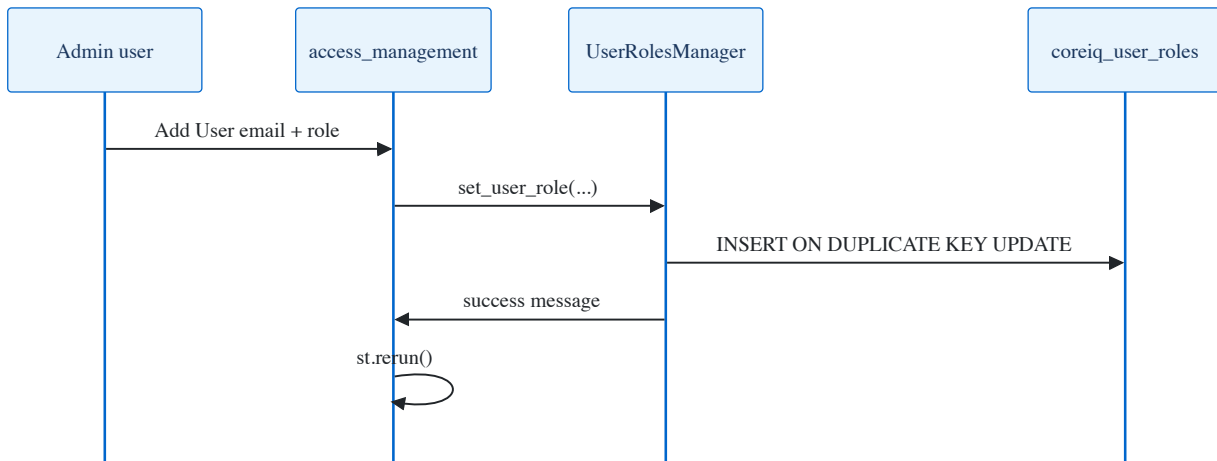
Users table

- Filter: All Roles / Admin / Super User / User
- Columns: Email, Display Name, Role (pill), Since, Added By, Delete
- **Delete flow:** Two-step confirm ( →)
- Cannot delete self
- Only admins can delete
- Calls `UserRolesManager.delete_user`

Role legend cards

Describes `admin` , `super_user` , `user` capabilities from `ROLE_DESCS` .

Describes admin, superuser, user capabilities from ROLEDESCS.



Tab 2 — Page Permissions

Page selector

Dropdown of `MANAGED_PAGES` → loads `AccessControlManager.get_page_users(sel_page_id)` .

Stats

Metrics: Total with Access, View Only, Edit, Delete (`delete_admin` count).

Grant Access form

Field	Notes
Email	Manual entry
Permission	<code>view_only</code> , <code>edit</code> , and <code>delete_admin</code> (admin only)
Quick-pick	Selectbox of known users from roles table

Super user limitation: Can grant `view_only` and `edit` only — not `delete_admin` .

Calls `AccessControlManager.set_user_access(sel_page_id, email, perm, current_user)` .

Current access table

Per user row:

- Email (read-only display)
- Permission selectbox — **auto-saves on change** (compares to current, calls `set_user_access`)
- Granted date, Granted by
- Revoke button → `remove_user_access`

Permission legend

UI exposes simplified 3-tier model:

UI label	DB value	Description
View Only	<code>view_only</code>	Read-only
Edit	<code>edit</code>	View + edit
Delete	<code>delete_admin</code>	Full admin on page

Legacy values (`allow` , `deny` , `upload_only`) still exist in DB and `get_permission_options()` but are collapsed in display pills via `PERM_DISPLAY` .

Database Health Check

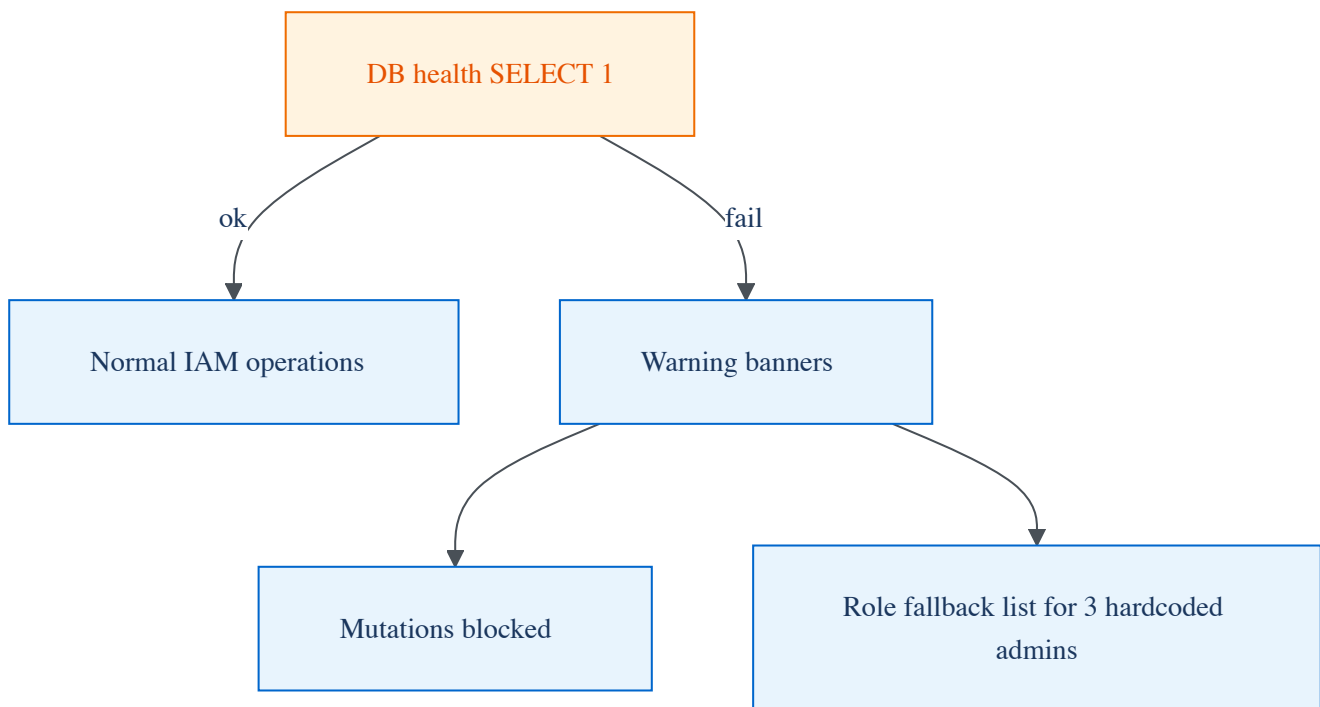
On page load:

```
db_manager.execute_query_readonly("SELECT 1", {})
```

If fails → `db_ok=False`:

- Warning banner on page
- Read operations show empty/cached states
- **All mutations disabled**
- Role fallback: hard-coded admin emails get `admin` role

- Role fallback: hard-coded admin emails get admin role



Session State Keys

Key pattern	Purpose
<code>am_u_email</code> , <code>am_u_name</code> , <code>am_u_role</code>	Add user form widgets
<code>am_u_filter</code>	Role filter selectbox
<code>_del_confirm_{email}</code>	Two-step delete confirmation
<code>am_p_sel</code>	Selected managed page index
<code>am_p_email</code> , <code>am_p_perm</code> , <code>am_p_pick</code>	Grant access form
<code>am_pp_{idx}_{email}_{page}</code>	Per-row permission selectbox

Role cache busting on write: `UserRolesManager` pops `_urm_role_{email}` from session.

Error Handling

Failure	UX
DB health check fail	Warning banner; mutations blocked; role list falls back to 3 hardcoded admins
<code>set_user_access</code> / <code>remove_user_access</code> exception	<code>st.error</code> + <code>[Access Control List (ACL)]</code> log line
Invalid email on add user	Inline validation before DB write
Non-admin attempting admin-only action	Button hidden or <code>st.error</code>

Page render wrapped in try/except at module level — failures log via `log_structured_error` with `page="access_management"` .

Logging

Uses `log_timing` for:

- `ACCESS_MGMT_auth`
- `ACCESS_MGMT_db_health`
- `ACCESS_MGMT_role_check`
- `ACCESS_MGMT_total_page_render`

ACL mutations log via `log_error` in `access_control.py` with `[Access Control List (ACL)]` prefix.

Relationship to Other Admin UIs

Surface	Overlap
<code>forecasting_admin.py</code> expander	Duplicate mini-IAM for 3 pages — uses same <code>AccessControlManager</code>
<code>admin_iam.py</code>	Unregistered legacy IAM page
Feature pages	Call <code>check_access</code> at module top

Canonical management: Access Management page is the primary full-featured IAM UI.

Admin Capability Matrix

Action	admin	super_user
Open page	Yes	Yes
Add user (any role)	Yes	Yes except admin role
Delete user	Yes	No
Grant view/edit page perm	Yes	Yes
Grant delete_admin page perm	Yes	No
Revoke page access	Yes	Yes

Fallback Admins (DB unavailable)

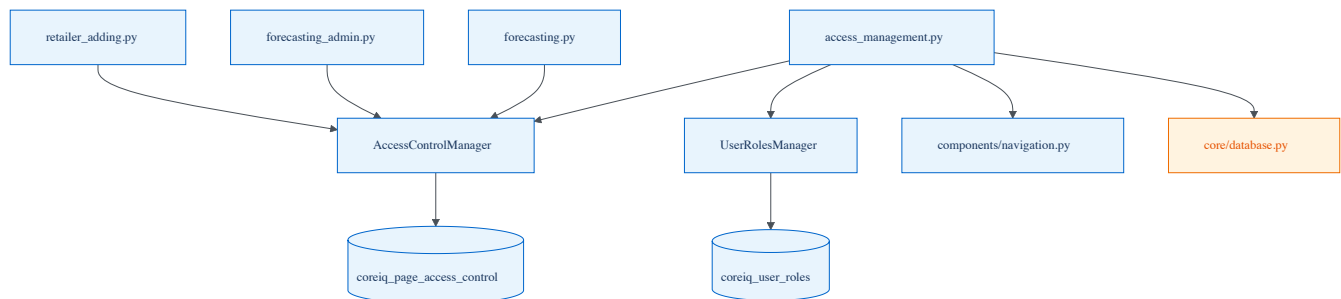
`UserRolesManager._FALLBACK_ADMINS` :

- mohdsaeedafri@coresight.com
- philipmoore@coresight.com
- shashankgupta@coresight.com

Also used in `_can_access_mgmt()` footer helper and Access Management role check fallback.

Dependency Graph

Dependency Graph



Generated from source analysis of the Market Data Portal (MDP). Last reviewed against codebase structure as of project documentation pass.